Costrutti di controllo e funzioni

Lezione 2 — 4MAK, SST-INF-01c — Firmware PIC: ripasso pre-stage

23/09/2026

1 Il costrutto `if`: decidere se eseguire del codice

I **costrutti fondamentali** di un linguaggio di programmazione sono quelli che permettono di decidere il **flusso esecutivo** del codice: **quali istruzioni** vengono eseguite, quante volte, e **in quale ordine**.

Definizione

Il costrutto `if` permette di **eseguire (o non eseguire)** un blocco di codice in base al valore di una **condizione**: un'**espressione booleana** che può essere **vera o falsa**.

```
if (condizione) {
    // codice eseguito solo se la condizione e' vera
}
```

La **condizione** si scrive tra parentesi tonde, usando **operatori di confronto** (==, !=, >, <, >=, <=) o **logici** (&&, ||, !). Il blocco di codice da eseguire va tra **parentesi graffe**.

Esempio

```
if (a == 5) {
    c = 7 * a;
    z = (a * 100 * 3) % a;
}
```

Se la variabile `a` vale 5, la **condizione** `a == 5` è vera e le **due istruzioni** dentro le graffe vengono eseguite. Se `a` vale, ad esempio, 7, la condizione è falsa e quel blocco di codice viene **saltato** interamente.

1.1 La variante `if-else`

Definizione

L'`if-else` aggiunge un **secondo blocco** di codice, eseguito quando la condizione è **falsa**:

```
if (condizione) {
    // eseguito se la condizione e' vera
} else {
    // eseguito se la condizione e' falsa
}
```

Con il **solo** **`if`** si sceglie se eseguire un blocco o saltarlo; con **`if-else`** si sceglie sempre **uno dei due** blocchi alternativi.

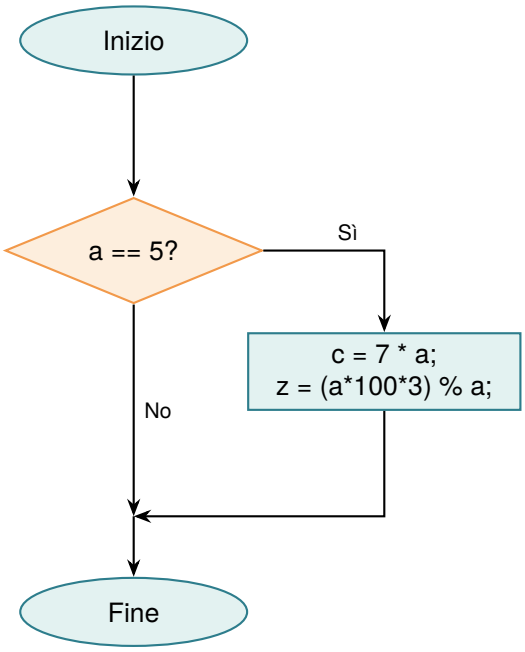


Figura 1: Diagramma di flusso dell'esempio `if (a == 5)`: il blocco con le due istruzioni viene eseguito solo se la condizione è vera, altrimenti il flusso salta direttamente al ricongiungimento.

Esempio

Un sistema che **innaffia una pianta** in base all'**umidità del terreno**:

```
if (umidita < 10) {
    // aziona la pompa: il terreno e' troppo secco
} else {
    // non azionare la pompa, il terreno va bene
}
```

Nota

Il **diagramma di flusso** è una **rappresentazione grafica** di questi costrutti: il codice arriva dall'alto, la condizione (un rombo) smista l'esecuzione su **due percorsi** — vero da un lato, falso dall'altro — che poi si **ricongiungono** per proseguire. Nell'`if` semplice, il percorso "falso" non contiene alcuna operazione: salta direttamente al ricongiungimento. Nell'**`if-else`**, invece, **entrambi** i percorsi contengono del codice da eseguire: sono i due diagrammi qui sotto, messi a confronto.

Diagramma dell'`if` semplice

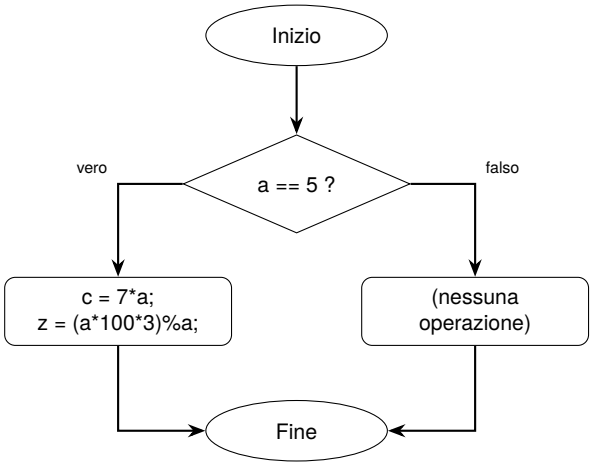
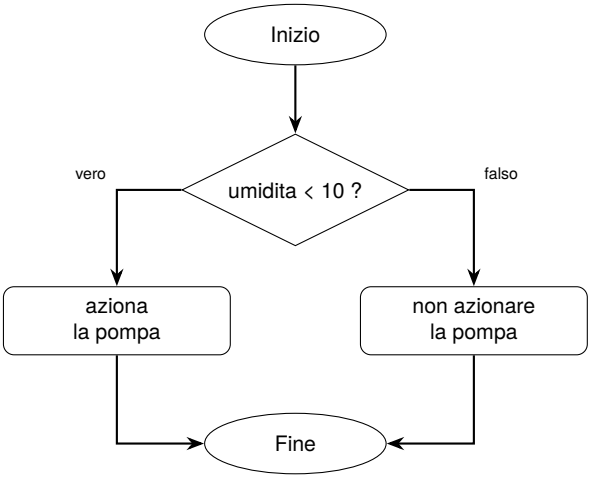


Diagramma dell'`if-else`



Attenzione

La differenza tra i due diagrammi è tutta nel **ramo "falso"**: nell'`if` semplice quel ramo è vuoto (non fa nulla, salta direttamente al ricongiungimento); nell'`if-else` quel ramo contiene un **blocco di codice alternativo**, sempre eseguito quando il primo non lo è. In entrambi i casi, dopo il rombo si esegue **sempre uno solo** dei due percorsi, **mai entrambi** e mai nessuno dei due.

2 I costrutti di ripetizione: `for`, `while`, `do-while`

Definizione

Le ripetizioni servono a eseguire **più volte** uno stesso blocco di codice. Esistono **tre costrutti**: **`for`**, **`while`** e **`do-while`**. Tutti e tre condividono lo stesso principio: finché **una condizione è vera**, si ripete. Cambia solo il modo in cui sono scritti, e in alcuni casi la comodità d'uso in base al problema.

Nota

Nessuno dei tre costrutti è strettamente indispensabile: qualunque ripetizione realizzabile con uno dei tre è replicabile (con qualche adattamento) anche con gli altri due. Si studiano comunque tutti e tre perché, lavorando in gruppo o in azienda, capiterà di **leggere codice scritto da altri** che usano lo stile che preferiscono.

2.1 Il ciclo `for`

Definizione

Il **`for`** raccoglie in un'**unica riga** tre elementi: **inizializzazione** (il **valore di partenza** del contatore), **condizione** di ripetizione, e **modifica** del contatore **a ogni giro**.

```
for (f = 0; f < 10; f++) {
    // codice ripetuto 10 volte
}
```

L'**operatore** `f++` è la forma abbreviata di `f = f + 1` (**incremento** di 1). Esiste anche `f--` per il **decremento**.

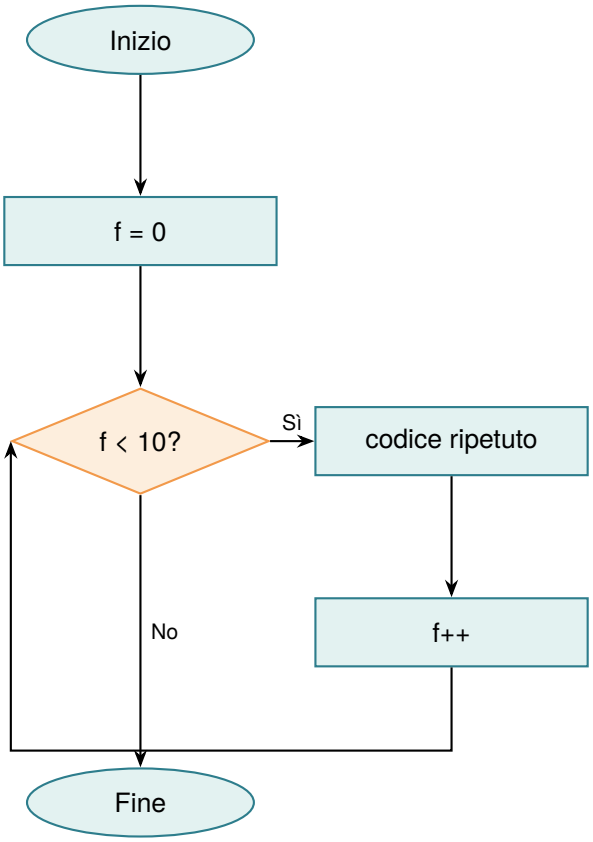


Figura 2: Diagramma di flusso del ciclo `for`: il rombo verifica la condizione prima di ogni giro; se vera si esegue il corpo e si incrementa il contatore, tornando al controllo; se falsa il ciclo termina e il flusso prosegue.

Esempio

Con `f` che parte da 0 e la condizione `f < 10`: la variabile assume i valori **0, 1, 2, ..., 9** — **dieci ripetizioni** in totale, perché quando `f` arriva a 10 la condizione `f < 10` diventa falsa e il ciclo si ferma.
Cambiando i parametri si ottengono **comportamenti diversi** con lo stesso costrutto: `for (f = 5; f < 15; f++)` produce ancora 10 ripetizioni, ma con `f` che va da 5 a 14. `for (f = 10; f > 0; f--)` conta **alla rovescia** da 10 a 1. `for (f = 0; f < 20; f += 2)` conta a **passi di 2**: 0, 2, 4, ..., 18.

Nota

Il valore assunto dal contatore a ogni ripetizione viene spesso usato **dentro** al ciclo per **parametrizzare i calcoli**: non è detto che ogni ripetizione debba eseguire esattamente le stesse operazioni con gli stessi numeri.

2.2 Il ciclo `while`

Definizione

Il **`while`** contiene **solo la condizione**; inizializzazione e modifica del contatore vanno scritte a parte, rispettivamente **prima del ciclo** e **dentro al blocco**:

```
f = 0;
while (f < 10) {
    // codice ripetuto
    f++;
}
```

A parità di inizializzazione, condizione e modifica, **`for`** e **`while`** eseguono **esattamente la stessa cosa**: cambia solo dove vengono scritti i tre elementi.

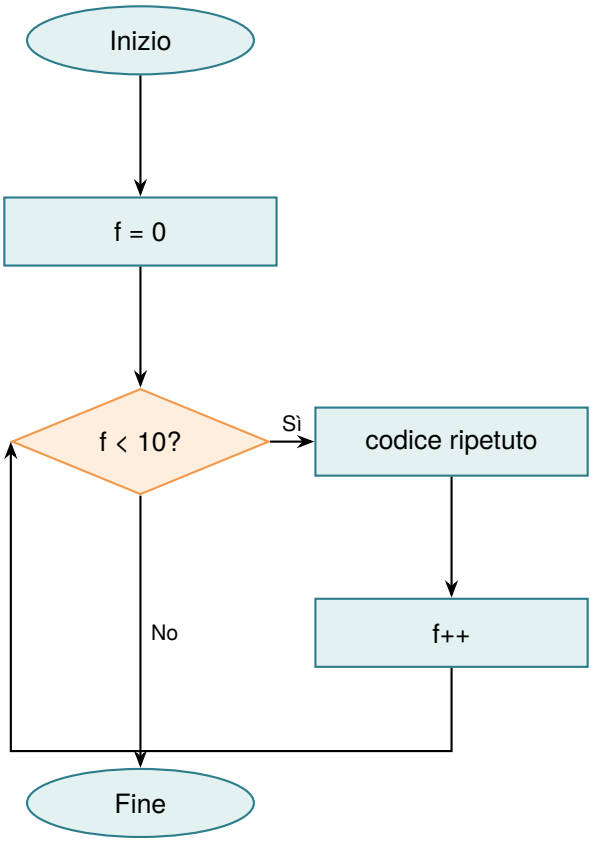


Figura 3: Diagramma di flusso del ciclo `while`: identico nella struttura al `for`, cambia solo dove sono scritti inizializzazione e incremento (fuori e dentro il blocco, invece che nella riga del ciclo).

Nota

Il **`for`** si preferisce quando il numero di ripetizioni è **noto a priori** (es. "ripeti 10 volte", "ripeti finché ho percorso 2000 impulsi di un motore stepper, e so già che sono 2000"). Il **`while`** si preferisce quando il numero di ripetizioni **dipende da un evento esterno** non prevedibile in anticipo (es. "continua a girare il motore finché il **sensore** non rileva un **ostacolo**", "continua a raffreddare finché la temperatura non scende sotto una **soglia**").

Esempio

Comunicare con un compagno bussando sulla parete: se ci si accorda che il segnale è **sempre** "**tre colpi**", il **numero di ripetizioni è fisso** e noto in anticipo — è un caso da **`for`**. Un robot che deve camminare fino in fondo alla stanza e fermarsi prima di sbattere contro il muro, invece, non sa a priori quanti passi servono: dipende da dove parte e da quanto è grande la stanza. In questo caso il numero di ripetizioni non è un valore fisso ma il **segnale di un sensore** che rileva l'ostacolo: è un caso da **`while`**.

Allo stesso modo, il **compressore di un frigorifero** deve restare acceso finché la **temperatura interna** non scende sotto una soglia (ad esempio 4°C): non si può **fissare un tempo** uguale in ogni stagione, perché in inverno bastano pochi minuti mentre in estate, con 40 gradi esterni, ce ne vogliono molti di più. Anche qui la ripetizione dipende da un evento esterno (la lettura della temperatura), non da un conteggio prestabilito: è un altro caso tipico da **`while`**.

Al contrario, un **motore stepper** che deve percorrere una **distanza nota** richiede un numero di impulsi **calcolabile in anticipo**: qui il **`for`** è la scelta naturale.

Nota

Nessuno dei tre costrutti di ripetizione è davvero indispensabile: si potrebbe scrivere qualunque programma usando solo il `for`, adattando gli altri casi con qualche trucco. È solo una questione di **comodità** e di stile: c'è chi, per anni, programma usando quasi solo il `for` e introduce il **`while`** molto più avanti, senza che questo gli abbia impedito di scrivere firmware funzionanti. Il **`do-while`**, in particolare, è l'unico dei tre di cui **si può fare a meno** più facilmente: tutto ciò che si fa con un `do-while` si può ottenere anche con un `while` scritto in modo leggermente diverso.

2.3 Il ciclo `do-while`

Definizione

Il `do-while` è una variante del `while` in cui il **controllo della condizione** avviene **alla fine** del blocco, invece che all'inizio:

```
do {
    // codice eseguito almeno una volta
    f++;
} while (f < 10);
```

Attenzione

La differenza pratica tra **`while`** e **`do-while`**: se la condizione è **già falsa al primo controllo**, il `while` non esegue mai il blocco, mentre il `do-while` lo esegue comunque **almeno una volta** prima di controllare la condizione.

Esempio

Un caso tipico di **`do-while`**: la **richiesta di una password**. Il codice deve chiedere la password **almeno una volta** prima di poterla verificare — non ha senso controllare una password

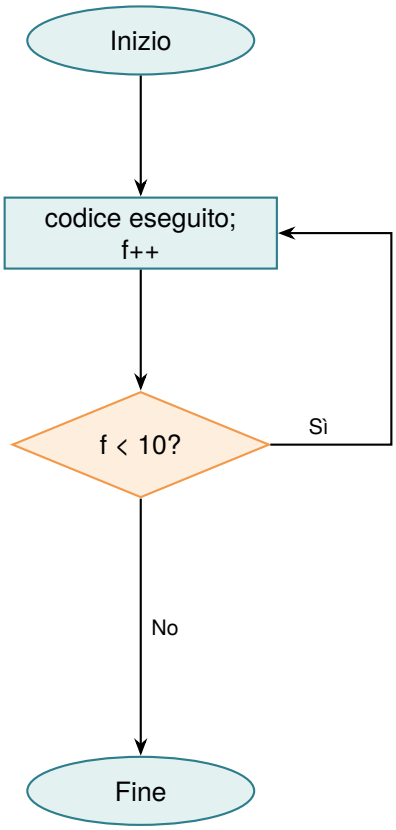


Figura 4: Diagramma di flusso del `do-while`: il blocco viene eseguito prima del controllo, quindi **sempre almeno una volta**; solo dopo si valuta la condizione per decidere se ripetere.

che non è ancora stata inserita. Si esegue il blocco (chiedi la password), poi si controlla la condizione (è corretta?), e **si ripete** finché non lo è.

3 Le funzioni

Definizione

Una **funzione** è un **pezzo di codice**, delimitato dalle solite **parentesi graffe**, a cui viene dato un **nome**. È concettualmente simile a una **macro**: una sequenza di operazioni richiamabile con un **unico "pulsante"** (il nome della funzione), invece di **riscrivere** ogni volta tutte le istruzioni.

Esempio

Un'analogia: immaginiamo una **funzione chiamata** `marco`, che rappresenta una persona che procura il pranzo. Se le do 12 euro (il **parametro** in ingresso), lei fa tutto il necessario – non mi interessa sapere come – e alla fine mi consegna il cibo (il **valore di ritorno**). Io, che chiamo la funzione, **non devo sapere cosa succede** "dentro" `marco`: mi basta **fornire il dato giusto** e **ricevere il risultato**.

Nota

Il vantaggio principale delle funzioni: se un calcolo richiede, ad esempio, **50 righe di codice**, raggrupparle sotto un unico nome permette di richiamarle con una sola riga ogni volta che servono, invece di riscriverle da capo. Questo vale anche per **problemi già risolti da altri**: una volta scritta una funzione corretta, la si **riutilizza** senza dover reinventare la soluzione ogni volta.

3.1 Tipo di ritorno e parametri

Definizione

Una funzione ha un **tipo di ritorno** (il tipo di dato che **restituisce**, scritto prima del nome) e può ricevere dei **parametri** (i dati che le vengono passati per svolgere il suo compito, tra **parentesi tonde** dopo il nome).

```
float areaCerchio(float raggio) {
    return raggio * raggio * 3.14159265;
}
```

Qui `float` prima del nome è il **tipo restituito**, `raggio` tra parentesi è il **parametro in ingresso**.

Esempio

Chiamando `areaCerchio(7)`, la funzione calcola $7 \times 7 = 49$, poi $49 \times 3{,}14159265 \approx 153{,}94$, e **restituisce quel valore** come `float` a chi l'ha chiamata.

Nota

Nei microcontrollori esiste un tipo speciale, `void`, che indica l'**assenza** di un valore di ritorno (una funzione che non restituisce niente) o l'**assenza di parametri**. A differenza di un programma per computer — dove esiste sempre un **sistema operativo** che lancia il programma e si aspetta una risposta finale — nel microcontrollore non c'è nessuno che "lancia" il codice dall'esterno: è il microcontrollore stesso ad avviarlo, quindi **non deve restituire nulla** a nessuno.

3.2 La funzione `main`

Definizione

In ogni programma deve esistere **una e una sola** funzione chiamata `main`: è la **funzione che il microcontrollore** (o il computer) esegue **per prima**, sempre. Dentro `main` si scrive tutto il codice che si vuole far eseguire, incluse le chiamate ad altre funzioni definite altrove.

```
void main(void) {
    // codice principale
}
```

3.3 Il codice non deve mai terminare: `while(1)`

Attenzione

Su un computer, quando la funzione `main` arriva alla fine, il programma **termina**: il **sistema operativo** lo chiude e torna disponibile per altro. Su un **microcontrollore** questo non deve succedere: non c'è un sistema operativo che gestisce la fine del programma, e se l'esecuzione arrivasse davvero alla fine di `main`, il comportamento della macchina diventerebbe **imprevedibile**. Un firmware deve girare **per sempre**, finché il dispositivo resta acceso.

Definizione

Per garantire che il codice non arrivi mai alla fine, si racchiude la parte di programma che deve ripetersi continuamente dentro un **ciclo infinito**, tipicamente scritto come **`while(1) { }`**:

```
void main(void) {
    // qui va il codice di configurazione iniziale,
    // eseguito una sola volta all'accensione

    while (1) {
        // qui va il codice che deve ripetersi
        // continuamente, finche' il dispositivo e' acceso
    }
}
```

Il numero **1** dentro le parentesi del `while` è **sempre "vero"** (ricordando che in informatica **qualunque valore diverso da 0** è vero): la condizione non diventa mai falsa, quindi il ciclo non si ferma mai. È un **ciclo infinito voluto**, non un errore.

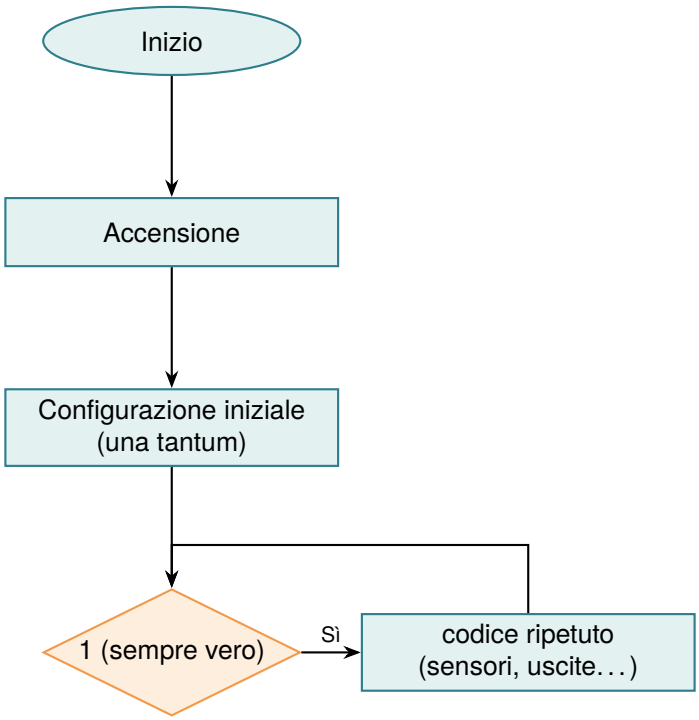


Figura 5: Diagramma di flusso dello scheletro base di un firmware: configurazione iniziale una tantum, poi ciclo infinito `while(1)` che non ha mai un'uscita verso il basso, perché la condizione è sempre vera.

Nota

Questa struttura – una parte di **configurazione iniziale** eseguita **una sola volta**, seguita da un **ciclo infinito** che gira per sempre – è lo **scheletro base** di praticamente ogni firmware per microcontrollore: prima si configurano i piedini, i registri, le periferiche (una tantum), poi si entra nel `while(1)` che contiene la logica che deve essere eseguita in continuazione (**leggere sensori**, aggiornare uscite, rispondere a eventi).

4 Cosa ricordare di questa lezione

- **if** esegue un blocco solo se **una condizione è vera**; **if-else** sceglie tra **due blocchi alternativi**.
- **for**, **while**, **do-while** fanno concettualmente la stessa cosa (ripetere finché **una condizione è vera**) ma con sintassi e comodità diverse: `for` quando il numero di ripetizioni è **noto a priori**, `while`/`do-while` quando dipende da un **evento esterno**.
- Il **do-while** garantisce **almeno un'esecuzione** del blocco prima del controllo della condizione — utile per input che vanno acquisiti prima di poterli validare.
- Una **funzione** raggruppa del **codice sotto un nome**, con un **tipo di ritorno** (o `void`) e **parametri** opzionali in ingresso.
- Ogni programma ha una sola funzione **main**: è il **punto di partenza** dell'esecuzione.
- Un firmware non deve **mai terminare**: il codice che deve ripetersi va dentro un ciclo infinito `while(1) { }`, dopo la **configurazione iniziale** eseguita una sola volta.